The ImGee UX Design Manifesto

Ergonomics, Cognitive Affordances, and Human Factors for Simulation User Interfaces

---

Authors & Architecture: Steven L. Goldberg (@slgoldberg) & the Antigravity Engineering Team

Custodian: ImGee Platform (ImGee / ImgWindow / LuiGee) • **Version:** v1.0.0 (Living Standard • Nov 2026)

Abstract

ImGee (pronounced *"im-jee"*, phonetic: `/ɪmˈdʒiː/`) is an opinionated, tactile C++ user-interface framework comprised of a self-consistent collection of standard Dear ImGui extensions, widgets, and layout primitives. It provides a modern, highly ergonomic foundation for building world-class 2-D user interfaces across a wide array of desktop applications.

Originally conceived and engineered for the flight simulation development community, ImGee was shaped by the unforgiving demands of high-cognitive-load, safety-critical cockpit environments. In simulated flight, pilots split attention between stick, throttle, instruments, and ATC. Clumsy interactions, vibrating tooltips, and ambiguous cursor affordances are fatal distractions that pull the user out of *Expert Mode* and force them into *Thinking Mode*.

While battle-tested in flight simulation, **ImGee is 100% generic Dear ImGui and is absolutely not restricted to X-Plane**. It contains zero hard dependencies on simulation SDKs or proprietary runtimes. Any desktop application, game-engine editor, digital audio workstation (DAW), robotics console, or scientific tool running Dear ImGui can consume ImGee directly.

Multi-Language Delivery & Ecosystem Topology

Bundled alongside **ImgWindow** (inside `contrib/ImGee/`) and architected as a standalone open-source library, ImGee provides a unified interaction vocabulary across multiple programming languages and runtime contexts:

- **C++ Native (`ImGee`):** Direct, zero-overhead immediate-mode C++11/C++17 integration for any Dear ImGui project.
- **Lua Scripting (`LuiGee`):** Turnkey Lua bindings for FlyWithLua (supporting X-Plane 11.20 through latest 12+ and beyond), bringing toggles, knobs, dials, and timed tooltips to script authors.
- **Future Language Bindings:** Architected so community or future bindings for other languages (such as Python, Rust, or C#) can share the exact same interaction standards.

Reverence for Dear ImGui & The Extensibility Model

ImGee was architected with profound respect for **Omar Cornut's** foundational vision for Dear ImGui.

Stock Dear ImGui deliberately remains a minimalist, unopinionated canvas -- primarily optimized for in-engine developer debuggers. ImGee honors this architecture by refusing to fork or mutate Dear ImGui internals, strictly following its official extensibility model (custom widgets, `ImDrawList`, layout stacks, font roles, ambient styles) to deliver:

1. **Active Cursor Affordances:** Banishing dead `Arrow` cursors in favor of explicit functional cues (`Hand`, `ResizeNS`, `ResizeEW`, `TextInput`).
2. **Stationary, Polite Assistance:** Replacing mouse-bound tooltip jitter with dwell-verified, stationary `TimedTooltips`.
3. **Physical vs. Categorical Consistency:** Enforcing boundary clamping for monotonic scalar values and circular wrapping for qualitative carousels.
4. **Natural Directional Geometry:** Guaranteeing that wheeling downward always advances downward (+1 index).
5. **Reversibility & Non-Destructive Defaults:** Equipping tunable controls with 1-click restore (↺) and instant undo.

Part I: The Architectural Foundation

The Framework's Promise to Developers & Users

Part I establishes the core human-factors principles, cognitive load theory, and structural interaction laws built directly into the ImGee framework engine.

> **NOTE**
>
> **Living Document & Design Intentionality:** This manifesto is not a static academic treatise or an immutable finished decree. It is an evolving architectural contract -- a compass designed to help developers and pilots understand where we are coming from, intuit how to use our work in the best possible way, and hold the ImGee framework accountable as our collective simulation experiences expand.

---

1. The Founding Premise: Cockpit Cognitive Reality

Flight simulation tools, glass-cockpit avionics, and mission utilities operate in a fundamentally different cognitive domain than web applications or office productivity software:

- **High Cognitive Load:** The pilot is actively flying the aircraft, trimming pitch, monitoring engine instruments, communicating with ATC, and maintaining situational awareness.
- **Peripheral & Fast-Action Modality:** Interactions with plugins and panels happen in quick, fleeting bursts -- often using one hand on a mouse while the other is on a flight stick or throttle.
- **The Golden Human Factors Rule:**

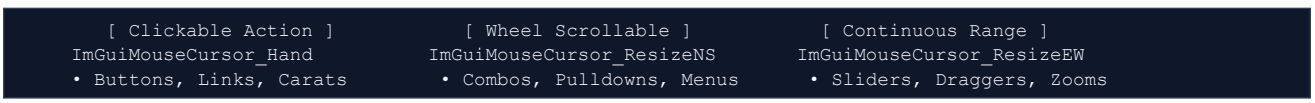
> ***Protect Expert Mode. Never force the user into Thinking Mode.***
>
> *In **Expert Mode**, the user operates entirely through spatial intuition, peripheral visual cues, and muscle memory. In **Thinking Mode**, the user is forced to pause, abandon aircraft control, squint at the UI, and diagnose why an element failed to respond to an input gesture. Every UI failure that triggers Thinking Mode is an architectural defect.*

Stock Dear ImGui is engineered for game-engine debuggers, where developers sit comfortably at desks inspecting memory trees with standard mice. ImGee re-engineers these foundations into an opinionated, tactile design language engineered specifically for simulation cockpits.

---

2. Cursor Affordances & The "Zero-Inert" Law

A mouse pointer is not merely an indicator of (X, Y) screen coordinates; it is the user's primary tactile feedback channel. In the absence of physical switch detents, the OS mouse cursor provides instant confirmation of what a widget is prepared to do.



The Zero-Inert Law

> ***An interactive, hot, or clickable widget must NEVER leave the mouse cursor on the stock `Arrow` (`ImGuiMouseCursor_Arrow`).***

- Leaving an interactive element on **`Arrow`** tells the user's subconscious that the control is passive text or dead space.
- Every interactive ImGee widget must declare its affordance explicitly:

1. `ImGuiMouseCursor_Hand` **(Point & Trigger):** Clickable pushbuttons, toggle rocker switches, hyperlinked text (`ButtonLink`, `TextLink`), and closed dropdown chevrons. Signals: *"Clicking this causes an action or toggles a menu."*
2. `ImGuiMouseCursor_ResizeNS` **(Vertical Cycle & Step):** Discrete multi-choice selectors, open dropdown option lists (`SelectPulldown`, `MouseWheelCombo`), and vertical value cyclers. Signals: *"Spinning the mouse-wheel will step through options."*
3. `ImGuiMouseCursor_ResizeEW` **(Continuous Horizontal Modulation):** Continuous range sliders (`SliderFloat`, `SliderInt`), drag controls, and zoom adjusters. Signals: *"Dragging horizontally or spinning the wheel smoothly modulates a continuous range."*
4. `ImGuiMouseCursor_TextInput` **(Text Entry):** Interactive text fields and scratchpads (`InputText`, `InputTextMultiline`).

---

3. The First-Gesture Discoverability Law

When an expert user encounters a new or unfamiliar dropdown or pulldown widget, they perform a **micro-gesture** -- a single notch of the mouse wheel -- to discover if the widget is interactive.

Natural List Order

> ***Scrolling the wheel DOWN (negative delta) MUST ALWAYS advance +1 down the visual list (0 → 1 → 2).***

```
  [ Item 0 (Charcoal) ]   ▲ Wheel UP   (-1 delta)
  [ Item 1 (Rusty)    ]   ▼ Wheel DOWN (+1 delta)
  [ Item 2 (Dark)     ]
```

The Initial-Gesture False Negative

If directional mapping is inverted under the hood (e.g. scrolling down decrements index rather than incrementing), an insidious cognitive failure occurs:

1. The user sees `Item 0` selected at the top of the list.
2. The user spins the mouse-wheel **down** to move to `Item 1`.
3. Because the internal logic decrements, it tries to select `-1`, which immediately hits the boundary clamp at `0`.
4. **Nothing moves on the screen.**
5. The pilot's brain reaches an immediate, completely logical, yet false conclusion: *"This widget does not support the mouse-wheel."*

The pilot never considers spinning *up* from the top item. Discoverability is destroyed on gesture one. Directional mapping must strictly mirror visual layout geometry across all platforms and bindings.

---

4. The Monotonic vs. Categorical Dialectic (Clamping vs. Wrapping)

Not all enumerable lists are created equal. When enabling mouse-wheel navigation, widget authors and developers must classify data into one of two fundamental categories:

A. Monotonic / Scalar Ranges (`wrap = false`)

- **Characteristics:** The options represent quantitative magnitude, physical distance, severity, or safety-critical states.
- **Examples:**
- Altimeter minimums, target distances (`0 m .. 1000 m`).
- Audio volume levels, display brightness (`0% .. 100%`).

- System power modes (`Off`, `Standby`, `Armed`, `Active`).
- **The Rule: Clamping at boundaries is mandatory.** Wrapping around from `1000 m` straight to `0 m` (or `Max` to `Off`) is disorienting, counter-intuitive, and in aviation software, potentially hazardous.
- **Ergonomic Requirement:** When hitting a boundary, the control must stop cleanly without overshoot.

B. Categorical / Carousel Sets (`wrap = true`)

- **Characteristics:** The options are qualitative presets, unordered modes, or cyclic vantage points where no intrinsic numeric floor or ceiling exists.
- **Examples:**
- Theme palettes (`Charcoal` → `Rusty` → `Dark` → `Classic` → `Light`).
- Camera views (`Cockpit 3D` → `Chase 30m` → `Tower View`).
- Navigation display modes (`Rose`, `Arc`, `Map`, `Plan`).
- **The Rule: Circular wrapping provides friction-free fluidity.** Hitting a dead end on an arbitrary palette or camera list forces the user to reverse wheel direction unnecessarily. Wrapping turns the control into an infinite carousel.

The Developer API Contract

Because a reusable widget cannot deduce the semantics of caller-provided strings at compile time:

- All ImGee enumerable widgets must default to safe clamping (`wrap = false`).
- All ImGee enumerable widgets must expose an explicit `wrap` parameter across all overloads:

```cpp
ImGui::SelectPulldown("##CameraMode", &s_CamMode, items, count, tooltips, width, anchor, align, /*wrap=*/true)
ImGui::MouseWheelCombo("##ThemePicker", &s_Theme, items, count, -1, /*wrap=*/true, tooltips);
```

---

5. The "Lens UI" Principle: Modal Focus in Navigation

When a dropdown or pulldown menu is opened, the user's interaction modality is fluid. They may click an item with the mouse, or they may keep the mouse stationary and spin the wheel to preview options.

Spatial Hover vs. Navigational Selection Focus

- When the mouse is moving across items, the active focus is **spatial hover**: the tooltip displays information about the row physically beneath the mouse cursor.
- The instant the user spins the mouse-wheel, their mental focus shifts to **navigational selection focus**: the user is actively driving the selection index (`*current_item`), and expects to inspect that newly selected item.

```
┌──────────────────────────┐
| Cockpit 3D               |  <- Mouse cursor physically resting here
| Chase 30m                |
| Tower View       [ACTIVE]|  <- Wheel spun down to here:
└──────────────────────────┘     TOOLTIP MUST LATCH TO TOWER VIEW!
```

The Latching Contract

1. When the mouse wheel steps `*current_item`, ImGee activates a **tooltip latch** (`s_WheelLatchPulldownID = id`).
2. While latched, tooltips target `items[*current_item]` dynamically, ignoring the stationary mouse pointer.
3. The user can rapidly spin the wheel through 10 items and read the context-sensitive tooltip for each successive selection in real time (the "Lens" effect).

The Hysteresis Threshold

- To prevent accidental unlatching caused by physical wheel vibrations (notched wheel detents often introduce a 1 -- 2 px micro-jiggle on physical mice), the latch must ignore micro-movements.
- The latch is broken only when deliberate pointer motion is detected:

```
[TEXT]
  Δx > 6.0 px   or   Δy > 6.0 px
```

- Once deliberate movement occurs, the system immediately reverts to standard spatial hover. Clicking an item cleanly clears the latch.

---

6. Container Scroll Pass-Through vs. Control Capture

In dense configuration windows, dataref inspectors, and checklists, multiple pulldown controls sit inside long vertically scrollable windows.

```
  [ Settings Window: Scrolling Down... ]
    ├─ Radio Nav Source:    [ NAV 1 ▼ ]  <- Closed: WHEEL PASSES THROUGH! Window scrolls.
    ├─ Transponder Mode:    [ ALT   ▼ ]  <- Closed: WHEEL PASSES THROUGH! Window scrolls.
    └─ Camera Preset:       [ CHASE ▲ ]  <- Open: WHEEL CAPTURED! Options cycle.
```

- **When Closed:** 1-line pulldowns (`SelectPulldown`) **must never capture the mouse-wheel**. Wheel gestures must pass cleanly through to the parent scrollable container. This guarantees pilots can scroll through a 50-row checklist or dataref inspector without accidentally altering operational settings.
- **When Open:** Once the menu is actively opened by the user, the popup claims modal focus. Inside the open popup window, mouse-wheel gestures are captured to cycle selection options dynamically.

---

7. The Law of Stationary, Polite Assistance: Why Stock Tooltips Fail and TimedTooltips Prevail

In a desktop simulator cockpit, tooltips should act like a calm, polite flight instructor: speaking only when asked, remaining completely composed, and stepping out of the way when work begins. Stock Dear ImGui tooltips (`ImGui::SetTooltip`, `ImGui::SetItemTooltip`) behave like the exact opposite: an agitated passenger waving papers directly across the pilot's scan.

```
      Stock Dear ImGui Tooltips                    ImGee TimedTooltips
  ──────────────────────────────────          ──────────────────────────────────
  • Fires instantly on 1-pixel brush          • Dwell timeout (500 ms) verifies intent
  • Bounces dynamically with mouse            • Anchored statically to widget bounds
  • Occludes adjacent controls & targets      • Floats clearly beside the active item
  • Forces user to swat mouse away            • Auto-dismisses on timeout or clean exit
  • Raw unformatted single strings            • Full CommonMark Markdown & status tags
```

The Three Deadly Defects of Legacy Tooltips

1. **The Instant-Flash Boundary Hazard (Zero-Dwell Strobing):**

Stock Dear ImGui displays tooltips on instantaneous hover. As a pilot swiftly traverses the screen to reach an autopilot disconnect button, the cursor sweeps across several intermediate knobs and checkboxes. Each brushed boundary instantly pops an opaque black window onto the screen for a few milliseconds, creating a strobe-like visual flicker. In a flight cockpit, peripheral visual flashes simulate emergency warning lights, triggering an involuntary reflex and disrupting the pilot's scan.

2. **The Mouse-Trailing Jitter Trap (Pointer Vibration):**

Stock tooltips are pinned to `io.MousePos`. As the user holds the mouse, subtle hand tremors, breathing, or micro-movements cause the tooltip window to vibrate and chase the cursor across the screen. Reading a multi-line

explanation while the text is vibrating is frustrating and visually exhausting.

3. **Target Occlusion:**

Because legacy tooltips cling directly to the mouse cursor, they invariably expand over the exact screen region the user is trying to inspect or click next, obscuring adjacent dials, switches, and readouts.

The ImGee TimedTooltip Standard

To resolve these defects, ImGee mandates the **Stationary TimedTooltip Architecture**:

- **Deliberate Dwell Verification:** Assistance is presented only after the cursor rests motionless over an element for a deliberate dwell threshold (default 500 ms). Rapid traversals across the cockpit produce complete visual tranquility.
- **Stationary Anchor Placement:** Tooltips are anchored solidly to the widget's physical bounding box (`ItemRectMin` / `ItemRectMax`). The text stays completely still, allowing the eye to read without chasing a moving target.
- **Polite Auto-Dismissal:** After an ergonomic reading duration (e.g. 8 -- 10 seconds), the tooltip cleanly dismisses itself. The pilot does not need to swat the mouse away to clear the panel.
- **Rich Markdown Formatting (`TimedTooltip::MD::Text`):** Tooltips are not restricted to plain text; they support formatted headers, bold labels, colorized status pills (`<pill>`), code blocks, and keyboard shortcut tags (`<kbd>`).
- **Non-Modal Interactive Chasers (`TextMDLink`):** Allows inline documentation links and expandable detail views that remain accessible without vanishing beneath the user's cursor.

---

8. Tactile Feedback, Reversibility, and Non-Destructive Defaults

Exploration encourages mastery. If users fear that adjusting a setting will permanently corrupt their configuration, they hesitate to interact.

- **1-Click Restore to Default (`SliderWithReset`, `ComboWithReset`):**
- Every adjustable slider or tuning combo must provide an inline reset button (↺) that restores the factory baseline in a single click.
- Clicking reset a second time acts as instant **Undo**, restoring the user's previous custom value.
- **Destructive Guardrails:**
- Modal file pickers (`SaveFileDialog`) must provide built-in file overwrite confirmation dialogs.
- Dangerous actions (e.g. wiping calibration profiles) require two-stage confirmation or long-press actuation.

---

9. Typography, Markdown, and Visual Signaling

Color and formatting are not decorative; in aviation interfaces, they are semantic status indicators.

- **Contrast Integrity:** Text must never blend into its background. If a style tag omits a foreground color (e.g. `<style bg=#444444>`), the rendering engine must calculate luminance and apply high-contrast text automatically.
- **Semantic Tagging Hierarchy:**
- **Capsule Tags (`<pill>`, `<badge>`, `<backdrop>`):** Define physical rounded boundaries with semantic background fills. Used for operational status (`<pill color=#2ecc71>ACTIVE</pill>`) and hardware states.
- **Text Style Tags (`<style>`, `<col>`, `<cam>`):** Modulate typography (font, weight, color) without injecting container boxes.
- **Template Aliases (`<tip>`, `<hotkey>`):** Provide turnkey, standardized visual stamps for operational guidance and keybindings.
- **Font-Scale Invariance:** Window chrome, status bars, and title bars must maintain a stable 1.0x baseline scale (`SetUnscaledChrome`), ensuring operational controls and indicators remain rock-solid regardless of internal document zoom.

Part II: The Developer's Covenant

Architectural Promises to End Users

The ImGee engine provides the physical affordances, but great user experience requires a partnership between the framework and the application developer. When you build avionics, tools, or plugins using ImGee or LuiGee, you make a sacred compact with your users to honor their cognitive airspace.

Here are the **Seven Promises** every developer must uphold:

---

1. The Tooltip Covenant: Banish Jitter and Flash

> *"I promise that I will never use legacy* `ImGui::SetTooltip()` *in my codebase again."*

- **The Commitment:**

I will use `ImGui::TimedTooltip::*` exclusively for all contextual guidance, or pass structured tooltip arrays directly into ImGee's turnkey controls (`SelectPulldown`, `MouseWheelCombo`, `ButtonTooltip`, `ToolbarToggleButton`).

- **Why It Matters to the User:**

Pilots sweep their mouse across your windows dozens of times per flight. By banishing instantaneous zero-dwell popups, you ensure your plugin never flickers in their peripheral vision or induces false warning reflexes. By anchoring tooltips statically, you ensure they can read your documentation with zero eye strain.

---

2. The Modality Covenant: Match Physics to the Data Variable

> *"I promise to never wrap monotonic physical values, and never dead-end categorical carousels."*

- **The Commitment:**

For every dropdown, combo, and stepped adjuster, I will conscientiously examine the data model:

- **Physical & Scalar Data (`wrap = false`):** Altimeter settings, radio frequencies, throttle steps, audio volumes, and sensitivity percentages will always clamp firmly at their endpoints.
- **Categorical & Cyclic Presets (`wrap = true`):** Themes, camera perspectives, visual modes, and liveries will wrap seamlessly into an infinite carousel.

- **Why It Matters to the User:**

Wrapping an altimeter or radio frequency from maximum back to zero in the middle of an approach is hazardous and jarring. Conversely, forcing a pilot to reverse wheel direction when browsing camera angles or color palettes adds pointless friction.

---

3. The First-Gesture Covenant: Honor Natural Reading Order

> *"I promise that scrolling DOWN will always advance +1 down the visual list."*

- **The Commitment:**

I will never invert mouse-wheel direction. I will verify that spinning the wheel downward advances downward through lists, menus, and selectable items in accordance with standard reading geometry.

- **Why It Matters to the User:**

When a user tests an unfamiliar control, their very first gesture is a single notch of the wheel downward. If the control decrements instead of incrementing, it immediately clamps against index 0. Nothing moves, and the user falsely concludes the widget is broken or unresponsive. Natural list order guarantees instant discoverability.

---

4. The Cursor Affordance Covenant: No Ambiguous Pixels

> *"I promise that no interactive element in my UI will ever leave the cursor on stock* `Arrow`*."*

- **The Commitment:**

Every clickable button, toggle, link, or stepped adjuster in my plugin will expose an active OS cursor affordance:

- Clickable actions & buttons: `ImGuiMouseCursor_Hand`
- Wheel-stepped lists & dropdowns: `ImGuiMouseCursor_ResizeNS`
- Continuous sliders & zoom thumbs: `ImGuiMouseCursor_ResizeEW`
- Text fields & editors: `ImGuiMouseCursor_TextInput`
- **Why It Matters to the User:**

The mouse cursor is the pilot's fingertip. Leaving an active control on `Arrow` makes it look like inert background decoration. Distinct cursor cues confirm at a glance what motor action is required before the user even clicks.

Host Integration & User Sovereignty (The Cursor Implementation Reality)

While the Zero-Inert Law is an absolute standard for widget design, the underlying Dear ImGui host architecture introduces real-world platform nuances that developers must actively navigate:

1. **Host-Level Cursor Support Varies:**

Not all Dear ImGui installations natively implement dynamic OS cursor changes. In some raw desktop backends or custom game-engine viewports, cursor routing is either unmapped or requires explicit host configuration (`io.ConfigFlags |= ImGuiConfigFlags_NoMouseCursorChange` overrides).

2. **Turnkey Support in ImgWindow v2:**

In the X-Plane ecosystem, ImGee is delivered alongside `ImgWindow v2`, which natively features a complete hardware and software cursor management subsystem. Custom cursors are supported out of the box -- both on a granular per-window basis (`SetUseCustomCursors(true)`) and globally across the plugin. Well-behaved plugins should ensure custom cursors are enabled by default so users immediately enjoy active visual feedback.

3. **User Sovereignty (The 1-Click Toggle):**

Users must always get the final say. Some simulation pilots perceive minor latency with software-rendered cursor overlays under heavy GPU loads, or simply prefer their OS pointer. Well-engineered plugins should provide an accessible preference toggle in their settings menu allowing users to disable custom cursor rendering (and re-enable it at will).

4. **Developer Responsibility: "Eat Your Own Dogfood":**

Developers should make every effort to keep custom cursors **enabled in their own development environments**. If you disable cursors while designing panels, you become blind to what end users experience by default. Keeping cursors active guarantees you will immediately catch regressions -- such as buttons erroneously hovering on inert `Arrow` or dropdown lists failing to switch to `ResizeNS`.

5. The Container Respect Covenant: Never Create Scroll Traps

> *"I promise that closed dropdowns and pulldowns will never hijack parent window scrolling."*

- **The Commitment:**

When using compact 1-line pulldowns (`SelectPulldown`) inside long scrollable settings dialogs or checklist windows, I will ensure wheel events pass cleanly through to the parent container when the control is closed.

- **Why It Matters to the User:**

Nothing is more aggravating than scrolling through a 40-item checklist only to have the scroll freeze because the mouse crossed a closed selector that silently swallowed the wheel gesture and changed an autopilot source. Closed controls must yield to the container; only open, focused menus capture modal input.

---

6. The Reversibility Covenant: Safe Defaults and 1-Click Restore

> *"I promise to never trap the user in a misconfigured state without an instant way home."*

- **The Commitment:**

For every tunable slider, audio control, or complex configuration parameter, I will provide a 1-click restore button (↺) that returns the value to its factory default, and supports immediate 1-click undo.

- **Why It Matters to the User:**

Simulation enthusiasts love to optimize and experiment, but fear of breaking a finely tuned setup inhibits exploration. A visible, reliable reset button invites confident experimentation, knowing perfection is always one click away.

---

7. The Information Hierarchy Covenant: Design for Peripheral Scanning

> *"I promise to design for rapid cockpit scanning, not dense walls of text."*

- **The Commitment:**

I will structure diagnostic data, checklists, and status indicators using ImGee's semantic typography:

- Use status pills (`<pill color=#2ecc71>ACTIVE</pill>`) for operational annunciators.
- Use keycaps (`<kbd>Ctrl+Shift+F12</kbd>`) for hotkey instructions.
- Use template aliases (`<tip>`, `<hotkey>`) for standard visual cues.
- Use Markdown tables (`ImGui::TextMD`) with column alignment for tabular flight parameters.

- **Why It Matters to the User:**

In a high-workload environment, pilots do not read sentences; they scan visual anchors. Structured formatting, distinct color-coded pills, and clean typographical weight allow the flight crew to comprehend system status in a 200 ms glance.

Appendix A: Canonical Verification Checklists

Quality Assurance & Compliance Standards

Before releasing any UI update or downstream plugin, verify the implementation against these two canonical quality standards:

Checklist A: Framework Verification (Holding ImGee Accountable)

- [ ] **Cursor State:** Does the widget display the mandatory non-arrow cursor when hovered?
- [ ] **Wheel Direction:** Does spinning the mouse wheel down advance +1 down the list (`natural_list_order = true`)?
- [ ] **Boundary Semantics:** Is `wrap = false` enforced for scalar ranges, and `wrap = true` supported for cyclic carousels?
- [ ] **Lens UI Latching:** While the dropdown is open, does spinning the wheel lock tooltip focus to `*current_item`?
- [ ] **Hysteresis Guard:** Does the tooltip latch resist mouse vibrations below the 6.0 px threshold?
- [ ] **Container Pass-Through:** When closed, does the pulldown allow wheel events to scroll the parent container?
- [ ] **TimedTooltip Integration:** Are all tooltips stationary, dwell-verified, and Markdown-capable?
- [ ] **Contrast Verification:** Does the styling engine guarantee legible text across light and dark backgrounds?

Checklist B: Developer Compliance (Holding Your Plugin Accountable)

- [ ] **Zero Legacy Tooltips:** Are all `ImGui::SetTooltip()` calls eliminated in favor of `TimedTooltip`?
- [ ] **Physical Value Clamping:** Are all physical parameters (altitude, frequency, volume) clamped at boundaries?
- [ ] **1-Click Restore Available:** Do tuning sliders and custom adjusters provide inline reset/undo buttons?
- [ ] **No Scroll Traps:** Can users scroll through dense configuration panels without tripping closed selectors?
- [ ] **Rapid Scanability:** Are critical status indicators rendered with semantic pills and badges rather than plain strings?
- [ ] **Scale Stability:** Are window chrome, buttons, and status bars anchored to unscaled baseline proportions?